

FORMAL REQUEST FOR CREDIT RESTORATION

Undisclosed Codex Auto Review activity, configuration override, and abnormal usage multiplication

To: OpenAI Billing Support and Codex Product Support

From: Jason | Account email: _____

Date: July 30, 2026

Subject: Request for full credit restoration and a server-side billing investigation

Prior case: Reference number, if applicable: _____

REQUESTED OUTCOME

Restore the credits consumed by the separately recorded Codex Auto Review activity and any abnormal main-task usage materially multiplied by that hidden review path; provide an itemized server-side ledger and explain why the runtime overrode my explicit reviewer setting.

Executive summary

On July 30, 2026, the OpenAI usage dashboard attributed 107 turns to a model labeled “Codex Auto Review,” separately from the GPT-5.6-SOL model I knowingly used for my Codex task. I did not select that separate model, request it, or knowingly consent to it as a separately metered source of usage. The activity consumed essentially all of my remaining credits.

A review performed through OpenAI’s own Codex product then located corresponding local session records. Those records show a second, hidden guardian-model path using the slug `codex-auto-review`. For the same UTC July 30 period, the local records contain exactly 107 guardian user-message events and 107 guardian agent-message events. That exact numerical reconciliation is strong evidence that the dashboard’s unfamiliar 107-turn category came from these background review sessions rather than from actions I intentionally initiated as separate model conversations.

The same review found that my global Codex configuration explicitly set `approvals_reviewer = “user,”` while the active task runtime repeatedly applied `approvals_reviewer = “auto_review.”` The available local records establish the mismatch and the resulting extra inference path. They do not reveal why OpenAI’s runtime overrode the setting, what server-side price was applied, or whether the behavior resulted from a rollout, managed policy, or software defect. Those are questions only OpenAI can answer from its internal records.

This is also not my first unexplained credit discrepancy. Prior concerns have not produced a satisfactory, itemized explanation. The combination of a hidden model path, an ignored user configuration, exact agreement with the unexplained dashboard turns, and material depletion of my balance warrants full credit restoration—not a generic explanation of cached tokens.

What happened

I used Codex Desktop for legitimate engineering work on a local repository. The visible task ran on GPT-5.6-SOL. During that work, the agent entered an excessive loop of small file reads, patch operations, command executions, and verification steps. Codex later acknowledged that the 84–125 model requests seen in several implementation slices were disproportionate to the project risk and were not necessary for safe, productive work.

At the same time, Codex Desktop’s runtime routed proposed tool actions through a separate guardian review path. Each action could therefore create another model inference in addition to the visible SOL task. This produced a multiplier: an overly granular agent workflow generated unusually many actions, and the product-level auto-review mechanism separately evaluated those actions.

WHY THIS MATTERS

The excessive tool loop was an agent behavior problem. Enabling a separately recorded background reviewer—despite a user-review configuration—was a product/runtime behavior. The financial impact arose from both layers interacting. Neither was a separate model workflow I knowingly chose.

Evidence from Codex’s local records

The following findings were derived from local Codex configuration, session JSONL records, and Codex’s model metadata. I am providing the relevant records or sanitized excerpts with this request. OpenAI should verify them against its server-side request and billing logs.

Finding	Locally observed evidence	Why it matters
Separate model path	Guardian subagent sessions identify model slug codex-auto-review; model metadata names it GPT-5.6-Terra and describes it as a balanced agentic coding model.	The dashboard label was not merely another name for the visible SOL task.
Exact turn match	UTC July 30 records contain 107 guardian user-message events and 107 guardian agent-message events.	This exactly matches the dashboard’s 107 Codex Auto Review turns.
Guardian role	Session metadata identifies the source as subagent / guardian and shows prompts evaluating proposed agent actions.	The activity was background approval review, not a conversation I opened.
Configuration mismatch	Global config: approvals_reviewer = “user.” Active task runtime: approvals_reviewer = “auto_review.”	My stated preference was overridden or ignored.
Material token volume	Guardian totals for UTC July 30: 3,826,716 input; 3,073,792 cached input; 9,444 output; 4,589 reasoning-output tokens.	The hidden path was substantial enough to have a meaningful account impact.
Abnormal visible-task loop	Several implementation slices used 84–125 model requests and 82–122 tool calls each.	The background reviewer multiplied an already excessive action pattern.

Important limitation: local token records do not disclose OpenAI’s server-side pricing or exact credit conversion. I therefore request a ledger rather than asserting an unsupported dollar calculation.

Exact local usage reconciliation

Local guardian measure — UTC July 30	Count
Guardian user-message events	107
Guardian agent-message events	107
Token-count events	110
Input tokens	3,826,716
Cached input tokens	3,073,792
Approximate non-cached input tokens	752,924
Output tokens	9,444
Reasoning-output tokens	4,589

Why credit restoration is warranted

1. **No informed selection or consent.** I knowingly selected and interacted with GPT-5.6-SOL. I did not knowingly select Codex Auto Review / GPT-5.6-Terra as a separately recorded model or authorize it to consume a material portion of my balance.
2. **My explicit configuration was not honored.** The global setting called for user review. The active runtime nevertheless applied automatic model review. A user should not bear the cost of a hidden model path that ran contrary to the available configuration.
3. **The unexplained dashboard category is independently corroborated.** The exact 107-event match ties the dashboard category to guardian sessions in Codex’s own local records. This is substantially more than a subjective impression that usage seemed high.
4. **The usage was magnified by abnormal product behavior.** The visible agent’s excessive action loop was then reviewed action-by-action by the hidden guardian path. The resulting multiplier was not a reasonable or transparent consequence of the engineering task I requested.
5. **The financial impact was material.** The activity consumed essentially all remaining credits available on my account, interrupting my work and forcing me to investigate OpenAI’s own internal behavior instead of completing the task for which I purchased the credits.
6. **This follows a prior unresolved credit discrepancy.** Because this is not my first unexplained usage issue, another generalized response would not restore confidence. I need a transaction-level explanation and a concrete remedy.

Requested remedies

Please treat this as a formal billing dispute and product escalation. I request the following:

1. **Restore the affected credits.** Refund or restore all credits charged to Codex Auto Review / guardian activity during the affected UTC July 30 window.

- 2. **Review the abnormal main-task charges.** Identify and restore any additional credits materially attributable to the runaway tool-call loop and its interaction with the hidden review multiplier.
- 3. **Provide an itemized server-side ledger.** For every affected request, provide timestamp, request or trace identifier, model slug, initiating task, input tokens, cached input tokens, output tokens, reasoning tokens, applicable price, and credits deducted.
- 4. **Explain the configuration override.** State why the active task applied `approvals_reviewer = "auto_review"` while the global configuration specified `"user,"` and identify whether the cause was a rollout, managed policy, account setting, application defect, or another mechanism.
- 5. **Provide a supported opt-out.** Give me a documented method to disable model-based automatic review permanently, confirm that it is effective for both new and existing tasks, and disclose any tradeoffs before additional metered use occurs.
- 6. **Preserve and investigate the records.** Preserve the server-side logs for the affected task and guardian sessions, reconcile them to the local evidence, and confirm whether other customers may have experienced the same configuration mismatch.
- 7. **Respond with a substantive determination.** Please do not close this with a generic explanation of caching, rate limits, or normal model usage. The disputed issue is the separately recorded guardian model, its exact 107-turn match, and the apparent override of a user-review configuration.

Prior unresolved discrepancy and trust impact

This incident has materially changed how I view the earlier unexplained credit issue on my account. Previously, I lacked enough evidence to distinguish an accounting misunderstanding from a product defect. Here, OpenAI’s own product located a hidden reviewer path, reconciled it exactly to the unfamiliar dashboard category, and identified a configuration mismatch. That evidence makes it unreasonable to dismiss the loss as ordinary usage without a full ledger.

The experience felt misleading and financially unfair. I do not allege that the available local records prove intentional misconduct; they do establish that I incurred material usage from behavior I did not knowingly select and that contradicted the reviewer setting visible in my configuration. Restoring the disputed credits and providing a transparent explanation is the minimum reasonable remedy.

Prior support case summary (optional):

Evidence file references

Evidence item	Local reference and relevance
Global user configuration	<code>~/.codex/config.toml</code> — <code>model = "gpt-5.6-sol"; approvals_reviewer = "user."</code>
Affected visible task	<code>~/.codex/sessions/2026/07/25/rollout-2026-07-25T17-42-13-019f9bdf-5714-7630-9fb3-f763ede951b8.jsonl</code> — runtime context repeatedly records <code>approvals_reviewer = "auto_review."</code>

Evidence item	Local reference and relevance
Example guardian session	~/.codex/sessions/2026/07/30/rollout-2026-07-30T18-18-35-019fb5c0-7067-7832-81fa-30c0dceb6493.jsonl — subagent/guardian source; model codex-auto-review.
Local model metadata	~/.codex/models_cache.json — codex-auto-review slug, GPT-5.6-Terra display name, and automatic-review policy metadata.
Historical comparison	~/.codex/sessions/2026/07/09/rollout-2026-07-09T20-10-12-019f4a01-1251-7da0-bf34-430ee876e078.jsonl — earlier user-review runtime context and later auto-review context.

Suggested attachments

- Screenshot of the OpenAI usage dashboard showing 107 Codex Auto Review turns separately from SOL.
- Screenshot or export showing the credit balance before and after the affected period.
- Sanitized excerpt of ~/.codex/config.toml showing approvals_reviewer = "user."
- Sanitized guardian-session metadata and UTC July 30 event/token totals.
- Sanitized affected-task runtime context showing approvals_reviewer = "auto_review."
- Any prior support correspondence or case number concerning unexplained credits.

Certification and response request

This statement is based on my account experience and local records generated by Codex Desktop. I have intentionally distinguished confirmed local evidence from questions that require OpenAI's server-side records. I request written confirmation that the dispute has been escalated to both billing and the Codex product team, together with the credit-restoration decision and the requested ledger.

Disputed credit amount or balance: _____

Preferred contact email: _____

Signature: _____

Date submitted: _____